Episode 4: Three Ways to Generate Network Configs

Video Title

"Network as Code: From YAML Intent to Device Configuration"

Target Length

20-25 minutes

Goal

Show the config generation pipeline in action. Demonstrate the route reflector auto-peering demo (the Cisco NaC paper's signature example). Compare Python/Jinja2, Ansible, and Terraform approaches without giving away the template code.

Pre-Recording Checklist

Before you hit record, make sure:
- Clean data model with all validations passing
- Generated configs exist in configs/ directory
- Ansible configs exist in configs-ansible/ directory
- Terraform configs exist in configs-tf/ directory
- Have spine1 and spine2's BGP config sections ready for the RR demo
- Know the exact line to toggle for the route reflector flag

SEGMENT 1: Recap and Framing (0:00 - 2:00)

What to show: Quick shot of the data model file tree and a passing validation run.

```
ls data/
uv run python validate.py
```

Talking points:
- "We have a data model. We have four layers of validation proving it is correct. Now we need to turn that intent into actual device configurations."
- "This is where the magic happens. You describe what the network should look like, and the automation figures out the exact CLI commands for each device."
- "Lab 3 does this three different ways: Python with Jinja2 templates, Ansible roles, and Terraform. I will show you all three and when each one makes sense."

SEGMENT 2: Python + Jinja2 (2:00 - 7:00)

What to show: Run the Python renderer and show the output.

```
uv run python -m generators.python.render
```

Show the output directory:

```
ls configs/
ls configs/spine1/
```

Show a snippet of generated config (NOT the template):

```
head -30 configs/spine1/frr.conf
```

Talking points:
- "The Python renderer reads the data model, builds a context for each device based on its role and the fabric design, and feeds that into Jinja2 templates."
- "Jinja2 is a templating engine. You write a template with placeholders, and it fills them in. The same template generates different configs for different devices based on their role."
- Show the head output: "This is the generated FRR config for spine1. Interface definitions, OSPF configuration, BGP neighbors. All of this came from the YAML data model."
- "I am not going to show you the templates themselves. That is in the build guide. But the concept is straightforward: one template per feature area. OSPF template, BGP template, VRF template. The renderer assembles them per device."

Quick comparison of spine vs leaf:

```
diff <(grep "neighbor" configs/spine1/frr.conf) <(grep "neighbor" configs/leaf1/frr.conf)
```

- "Look at the difference. Spine1 has route-reflector-client on every neighbor statement. Leaf1 just has the two spines as neighbors. Same templates, same data model, different output based on role."

SEGMENT 3: The Route Reflector Demo (7:00 - 12:00)

This is the money demo for this episode. Take your time.

What to show: The auto-peering behavior when you change a single flag.

Step 1: Show the current state

```
grep -n "route.reflector\|neighbor" configs/spine1/frr.conf | head -15
grep -n "route.reflector\|neighbor" configs/spine2/frr.conf | head -15
grep -n "neighbor" configs/leaf1/frr.conf | head -10
```

- "Right now, both spines are route reflectors. Every leaf peers with both. Standard design."

Step 2: Change one flag

Exact edit: In data/topology.yaml, change line 21 from route_reflector: true to route_reflector: false

- "I am changing one flag. spine2's route_reflector from true to false. One line in one YAML file."

Step 3: Regenerate

```
uv run python -m generators.python.render
```

Step 4: Show the result

```
grep -n "route.reflector\|neighbor" configs/spine1/frr.conf | head -15
grep -n "route.reflector\|neighbor" configs/spine2/frr.conf | head -15
grep -n "neighbor" configs/leaf1/frr.conf | head -10
```

Talking points:
- "Look at what happened. Spine1 gained spine2 as a route-reflector-client, because spine2 is now just a regular iBGP peer. Spine2 lost all its route-reflector-client statements. Every leaf still peers with both spines, but the relationship changed."
- "Six device configs changed from one line in YAML. That is the power of intent-based automation. You do not manage configs. You manage intent, and the configs follow."
- "This is the exact demo Cisco uses in their Network as Code paper to explain why data models matter. And it works because the templates are role-aware. They look at the route_reflector flag and build the neighbor statements accordingly."

Step 5: Revert

Exact edit: In data/topology.yaml, change line 21 back from route_reflector: false to route_reflector: true, then regenerate:

```
uv run python -m generators.python.render
```

SEGMENT 4: Ansible Path (12:00 - 16:00)

What to show: The Ansible approach to the same problem.

```
ls generators/ansible/
ls configs-ansible/
```

Talking points:
- "Same data model, different tool. The Ansible path uses roles mapped to device roles. There is a spine role, a leaf role, and a border role."
- "Ansible uses the same Jinja2 under the hood, but the workflow is different. Instead of a Python script that builds context and renders templates, you have an Ansible playbook that assigns roles to hosts and lets Ansible's variable system handle the data."

Show the Ansible generated output:

```
head -30 configs-ansible/spine1/frr.conf
```

- "Same output. Different path to get there. If your team already runs Ansible for server automation, this might be the more natural fit."

Quick diff:

```
diff configs/spine1/frr.conf configs-ansible/spine1/frr.conf
```

- "Nearly identical output. The differences, if any, are cosmetic. Same intent, same result, different tooling."

SEGMENT 5: Terraform Path (16:00 - 19:00)

What to show: The Terraform approach.

```
ls generators/terraform/
ls configs-tf/
```

Talking points:
- "Terraform takes a fundamentally different approach. It is declarative and stateful. It tracks what it has created and what needs to change."
- "For config generation, we use Terraform's local_file provider. It reads the data model and writes config files, same as the other two paths. But Terraform also maintains a state file that knows exactly what it last generated."
- "That state awareness becomes valuable in Lab 6 when we talk about drift detection. Terraform can tell you 'the config file on disk does not match what I last generated' without you writing any comparison logic."
- "The trade-off is complexity. Terraform adds state management, provider dependencies, and a different language (HCL). For pure config generation, Python or Ansible is simpler. But if your organization already uses Terraform for infrastructure, extending it to network configs is a natural fit."

Show the Terraform output:

```
head -30 configs-tf/spine1/frr.conf
```

SEGMENT 6: When to Use What (19:00 - 21:00)

What to show: Simple comparison slide or just talking to camera.

Talking points:
- "So which one should you use? It depends on your team."
- "Python plus Jinja2: Maximum control, maximum flexibility. You understand everything that is happening because you wrote it. Best for teams that want to own their tooling."
- "Ansible: Great if your team already uses Ansible. The role-based structure maps cleanly to network device roles. Good community support for networking modules."
- "Terraform: Best when state management matters. If you need to track what was deployed and detect drift natively, Terraform's state model is purpose-built for that. Also the right choice if your organization has standardized on Terraform for everything else."
- "In this series, I use the Python path as the primary workflow. But the code for all three is in the lab package, and you can compare them side by side."

SEGMENT 7: The Close (21:00 - 23:00)

What to show: Series overview or back to camera.

Talking points:
- "That is Lab 3. Data model in, device configs out. Three different tooling paths, all producing the same result."

- "The build guide walks through creating every template, writing the render engine, and setting up the Ansible roles and Terraform configuration. Available at [your link] for $49."
- "Next episode, we wire this into a CI/CD pipeline. Push a feature branch, validation runs automatically. Open a PR, you get a config diff as a comment. Merge to main, configs deploy to the network. That is Lab 4."
- "Subscribe, and I will see you in the lab."

Commands Reference (Quick Copy)

```
# Python render
uv run python -m generators.python.render

# Show generated configs
ls configs/
head -30 configs/spine1/frr.conf

# Grep for BGP neighbors
grep -n "route.reflector\|neighbor" configs/spine1/frr.conf | head -15

# Diff Python vs Ansible output
diff configs/spine1/frr.conf configs-ansible/spine1/frr.conf

# Ansible configs
ls configs-ansible/

# Terraform configs
ls configs-tf/
```

Do NOT Show On Camera

- Jinja2 template source code (paid content)
- The Python render engine internals (paid content)
- Ansible playbook and role structure in detail (paid content)
- Terraform HCL configuration (paid content)
- Step-by-step process of building templates (paid content)

DO Show On Camera

- Generated config output (the result, not the template)
- The route reflector auto-peering demo (changing one flag, seeing six configs update)
- grep/diff output comparing configs between devices and between tooling paths
- File tree of each generator path
- The one line being changed in the RR demo (but not the full file)